REAL TIME DEVSECOPS BINGO APP PROJECT| Infrastructure WITH TERRAFORM AND JENKINS CICD PIPELINE

Project was to learn how to use web sockets and how to create a multiplayer game. It was created using Next.js, React and Sockets.io

Jenkins user = bingouser

Password = bingouser

Sonarqube = bingouser password

Jenkins integration

squ_02c240a69b9d0045e7043b3f38ce3917c73cad4e

```
sonar-scanner \
 -Dsonar.projectKey=Bingo \
 -Dsonar.sources=. \
 -Dsonar.host.url=http://65.0.102.160:9000 \
 -Dsonar.login=squ_02c240a69b9d0045e7043b3f38ce3917c73cad4e
```

- Enter the room with the new details

RoomId : testgame

Your Name : milky

Client side :

Chat ID , opens with RoomID and Name

Server Side :
In the application backend logs creates successfully

```
wait  - compiling...
wait  - compiling /_error (client and server)...
wait  - compiling...
event - compiled client and server successfully in 944 ms (210 modules)
wait  - compiling /[room]...
wait  - compiling...
event - compiled client and server successfully in 259 ms (218 modules)
wait  - compiling /api/socket...
wait  - compiling...
event - compiled client and server successfully in 210 ms (228 modules)
{ room: 'testgame', name: 'aseem' }
{ room: 'testgame', name: 'aseem', id: 'BnwZv-d63uMWXXQlAAAf' }
{ room: 'testgame', name: 'akram' }
{ room: 'testgame', name: 'akram', id: 'jjENzJN0sI8fmDAMAAAj' }
```

Login with RoomID with your another user

http://localhost:3000/testgame?name=milky

Table of contents

Step1: Create directory

Step2: Aws Configure

Step3: Terraform files and Provision Jenkins,sonar

Step 4 — Install Plugins like JDK, Sonarqube Scanner, NodeJs, OWASP Dependency Check

4A — Install Plugin

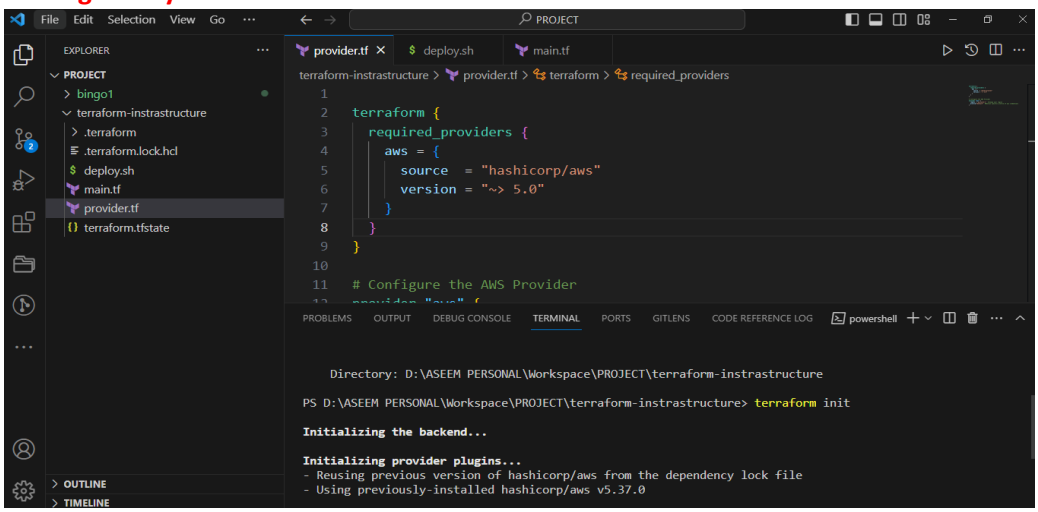
4B — Configure Java and Nodejs in Global Tool Configuration

Step 5 — Configure Sonar Server in Manage Jenkins

Step 6 — Install OWASP Dependency Check Plugins

Step 7 — Docker Image Build and Push

1. **Testing locally**



terraform apply

Prompt you to confirm the apply to create resource , enter the value “yes”

Or else use command to create without prompting the action
terraform apply --auto-approve

```
      + name                   = "Project-sg"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags                   = {
          + "Name" = "Project-sg"
        }
      + tags_all               = {
          + "Name" = "Project-sg"
        }
      + vpc_id                 = (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.Project-sg: Creating...
```

```
PS D:\ASEEM PERSONAL\Workspace\PROJECT\terraform-instrastructure> terraform apply --auto-approve
aws_security_group.Project-sg: Refreshing state... [id=sg-03f8d527f5fb1e6c2]

Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.webproject will be created
  + resource "aws_instance" "webproject" {
      + ami                                  = "ami-03f4878755434977f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
```

```
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = [
          + "sg-03f8d527f5fb1e6c2",
        ]

      + root_block_device {
          + delete_on_termination = true
          + device_name           = (known after apply)
          + encrypted             = (known after apply)
          + iops                  = (known after apply)
          + kms_key_id            = (known after apply)
          + throughput            = (known after apply)
          + volume_id             = (known after apply)
          + volume_size           = 24
          + volume_type           = (known after apply)
        }
    }

  Plan: 1 to add, 0 to change, 0 to destroy.
  aws_instance.webproject: Creating...
  aws_instance.webproject: Still creating... [10s elapsed]
  aws_instance.webproject: Still creating... [20s elapsed]
  aws_instance.webproject: Creation complete after 30s [id=i-0b3ac87f1116b0c66]

  Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Resources has been created validate with IP:8080 for jenkins

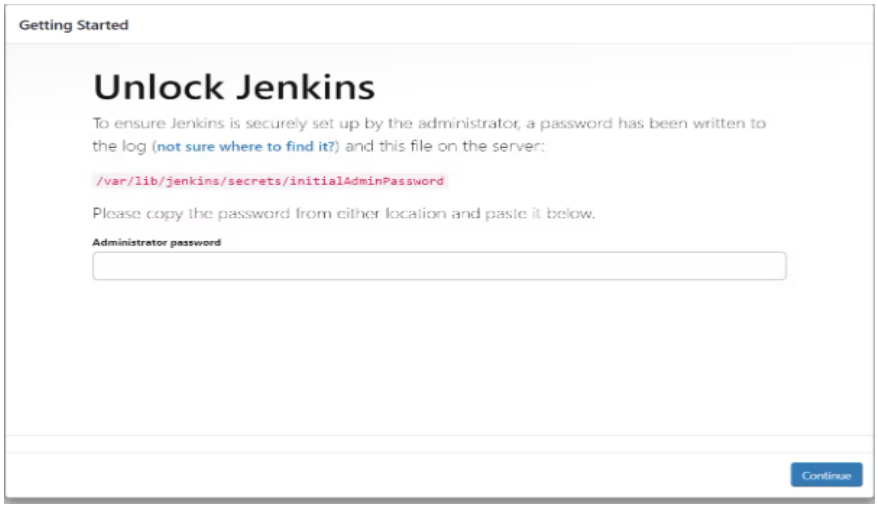
#Login to server with credentials , set the hostname with following command

Resources has been created validate with IP:8080 for jenkins

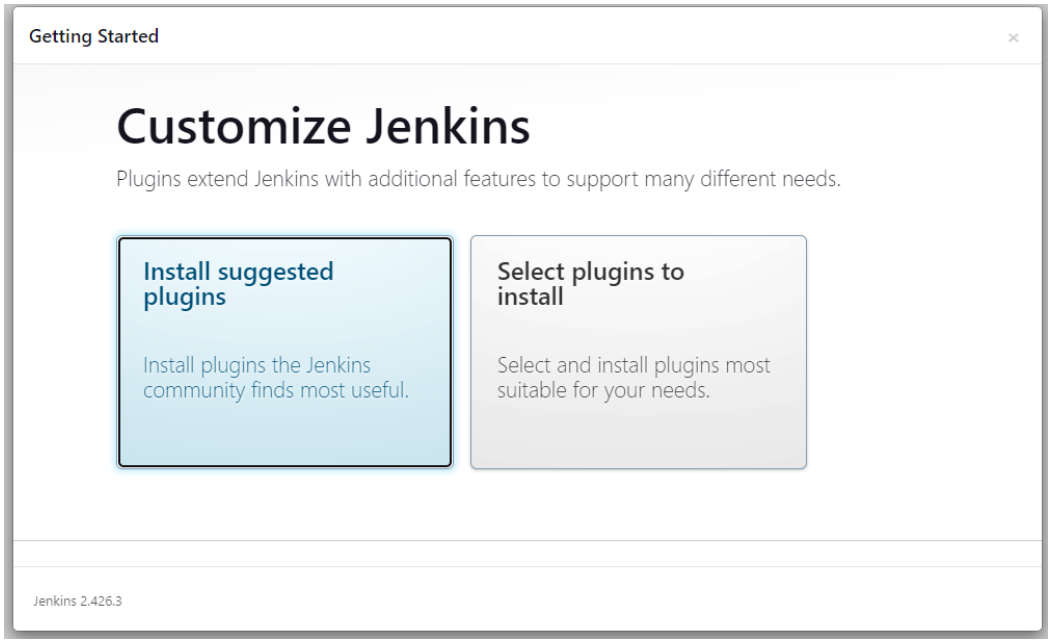
1. Jenkins

```
lines 1-20/20 (END)...skipping...
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2024-02-19 14:02:01 UTC; 3h 8min ago
   Main PID: 3239 (java)
      Tasks: 48 (limit: 9498)
     Memory: 2.1G
        CPU: 1min 8.960s
     CGroup: /system.slice/jenkins.service
             └─3239 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --we

Feb 19 14:01:42 ip-172-31-15-248 jenkins[3239]: 7502d862796c4081a684cef677dcadd3
Feb 19 14:01:42 ip-172-31-15-248 jenkins[3239]: This may also be found at: /var/lib/jenkins/secr
Feb 19 14:01:42 ip-172-31-15-248 jenkins[3239]: *************************************************
Feb 19 14:01:42 ip-172-31-15-248 jenkins[3239]: *************************************************
Feb 19 14:01:42 ip-172-31-15-248 jenkins[3239]: *************************************************
Feb 19 14:02:01 ip-172-31-15-248 jenkins[3239]: 2024-02-19 14:02:01.360+0000 [id=31]        INFO
Feb 19 14:02:01 ip-172-31-15-248 jenkins[3239]: 2024-02-19 14:02:01.374+0000 [id=24]        INFO
Feb 19 14:02:01 ip-172-31-15-248 systemd[1]: Started Jenkins Continuous Integration Server.
Feb 19 14:02:02 ip-172-31-15-248 jenkins[3239]: 2024-02-19 14:02:02.111+0000 [id=49]        INFO
Feb 19 14:02:02 ip-172-31-15-248 jenkins[3239]: 2024-02-19 14:02:02.112+0000 [id=49]        INFO
```

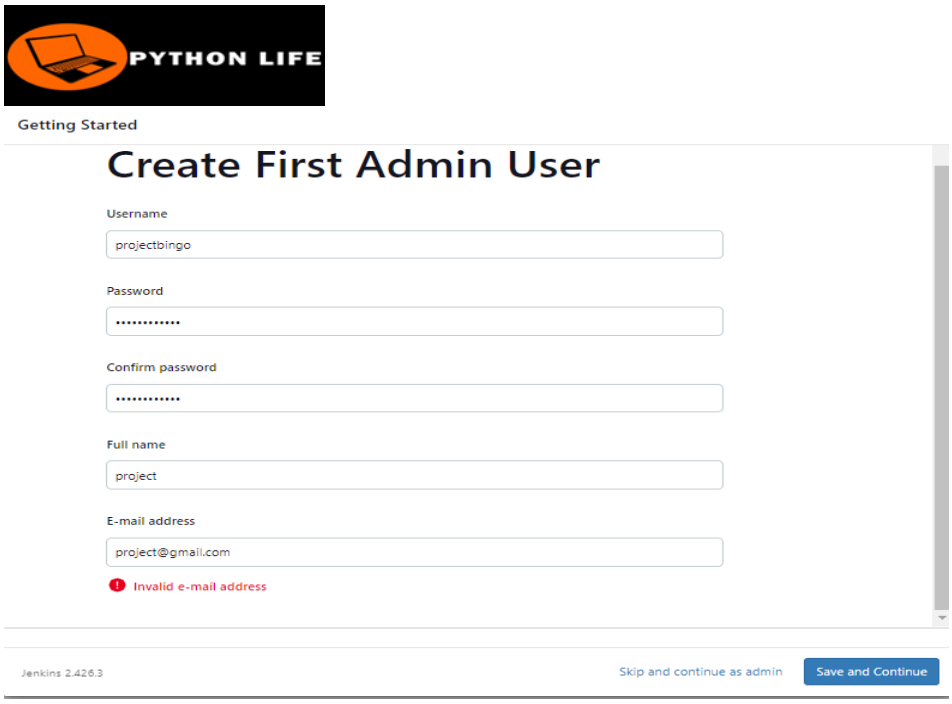


sudo cat /var/lib/jenkins/secrets/initialAdminPassword



Click on install suggested plugin default.

Create a user click on save and continue.

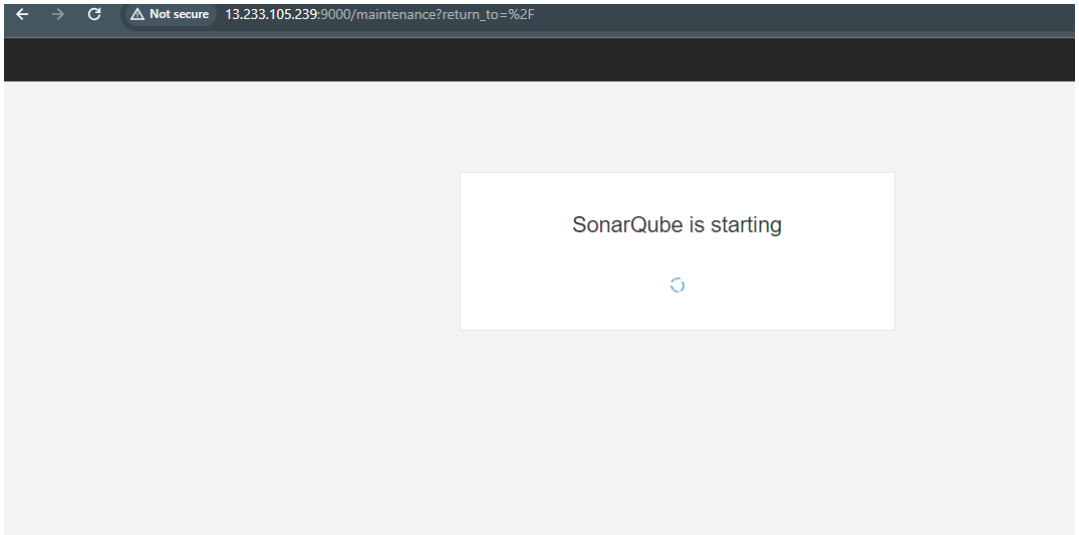


projectbingo as new password

Copy your Public key again and paste it into a new tab

<instance-public-ip:9000>

Now our sonarqube is up and running

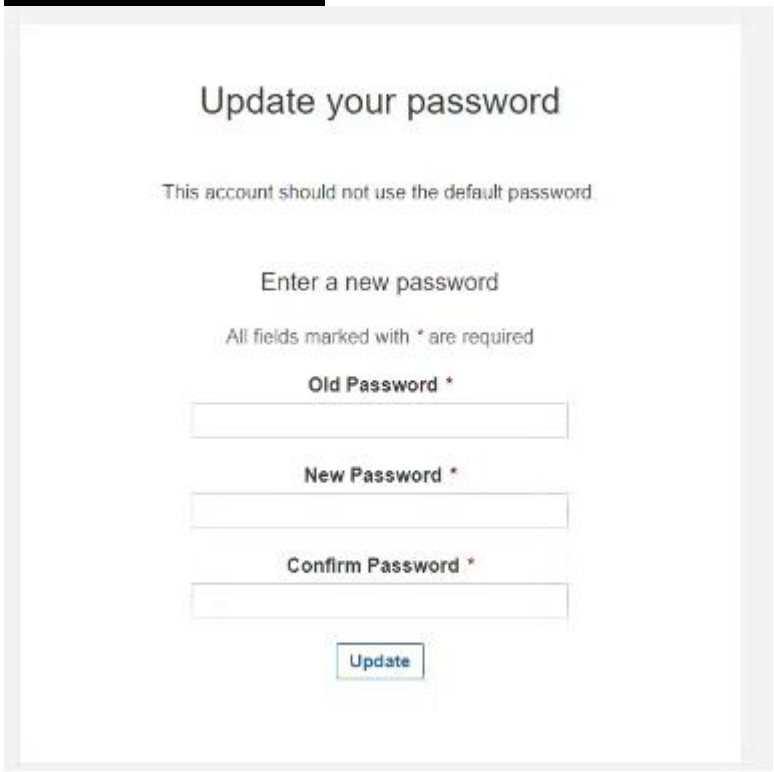


Enter Default username and password, click on login and change password

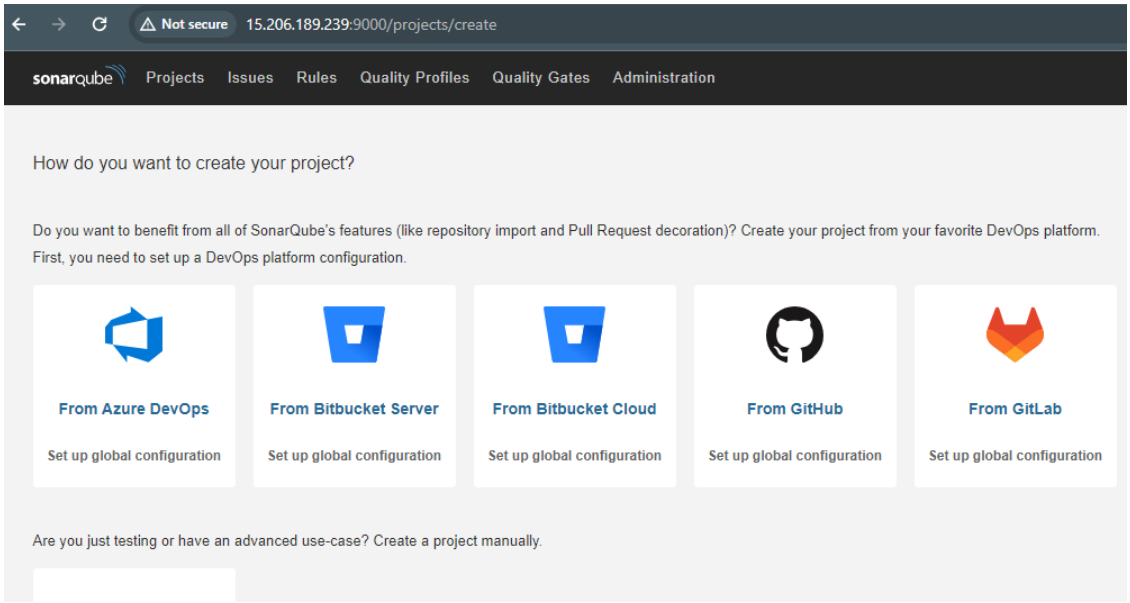
username admin

password admin

projectbingo as new password



projectbingo as new password

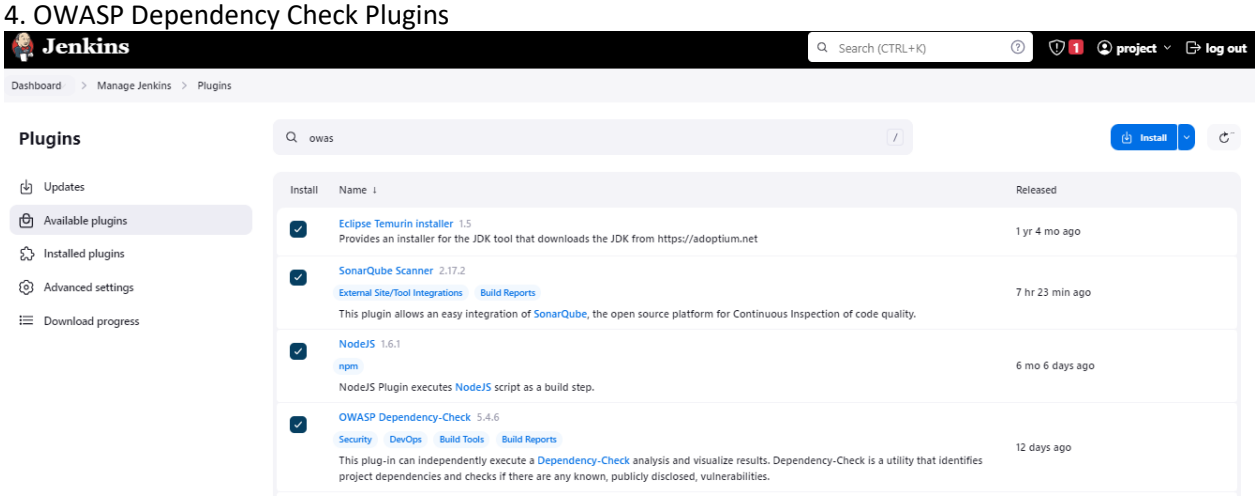


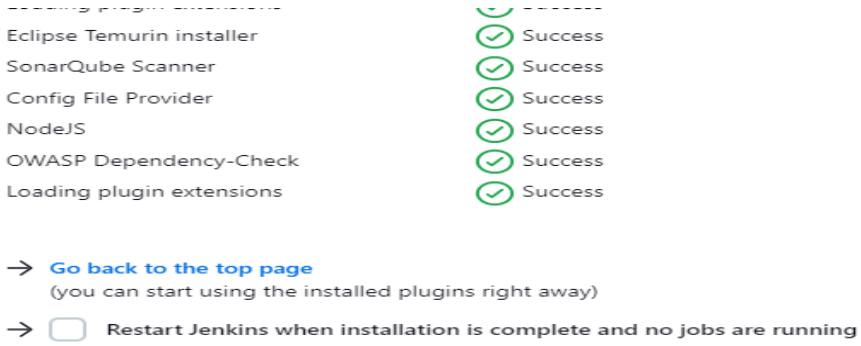
Check trivy version

```
root@bingo:~# trivy --version
Version: 0.49.1
root@bingo:~#
```

Install Plugins like JDK, Sonarqube Scanner, NodeJs, OWASP Dependency Check

Goto Manage Jenkins →Plugins → Available Plugins →
Install below plugins
1 . Eclipse Temurin Installer (Install without restart)
2 . SonarQube Scanner (Install without restart)
3 . NodeJs Plugin (Install Without restart) – 16.20.2
4. OWASP Dependency Check Plugins

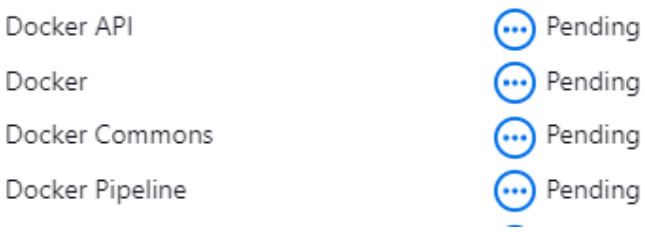




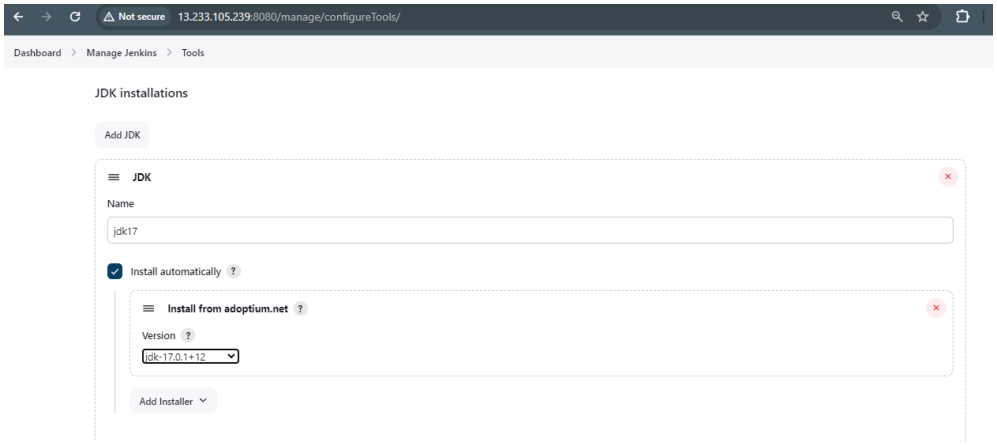
We need to install the Docker tool in our system, Goto Dashboard → Manage Plugins → Available plugins → Search for Docker and install these plugins

1. Docker
2. Docker Commons
3. Docker Pipeline
4. Docker API
5. docker-build-step

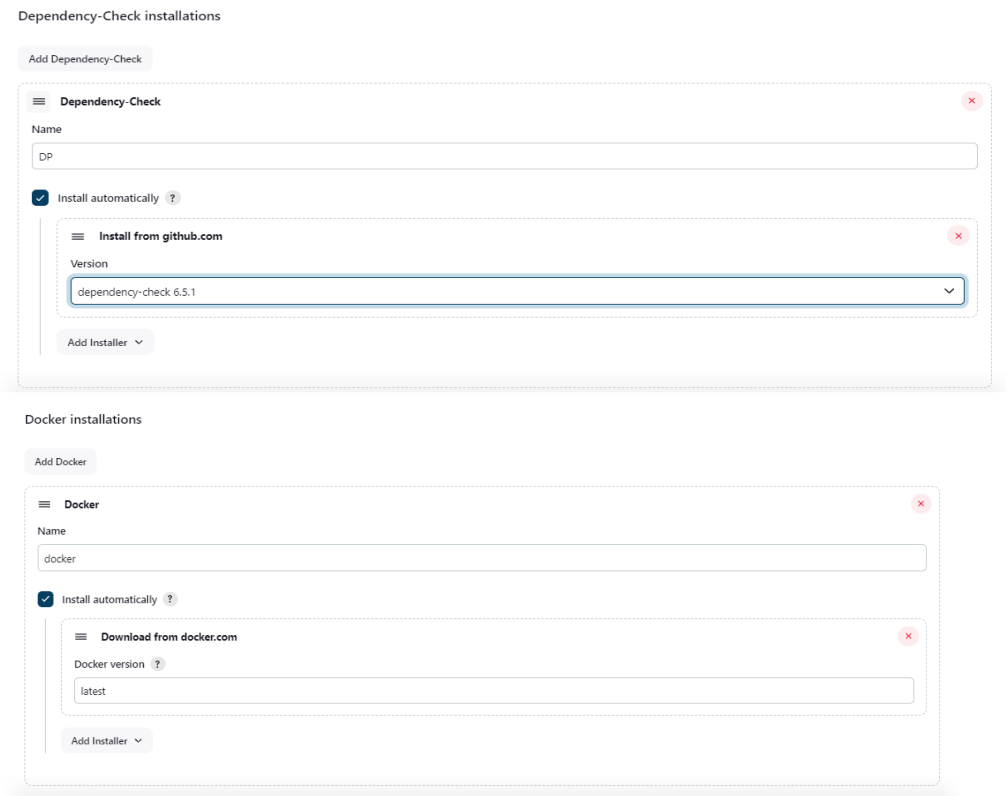
and click on install without restart



Now, goto Dashboard → Manage Jenkins → Tools →



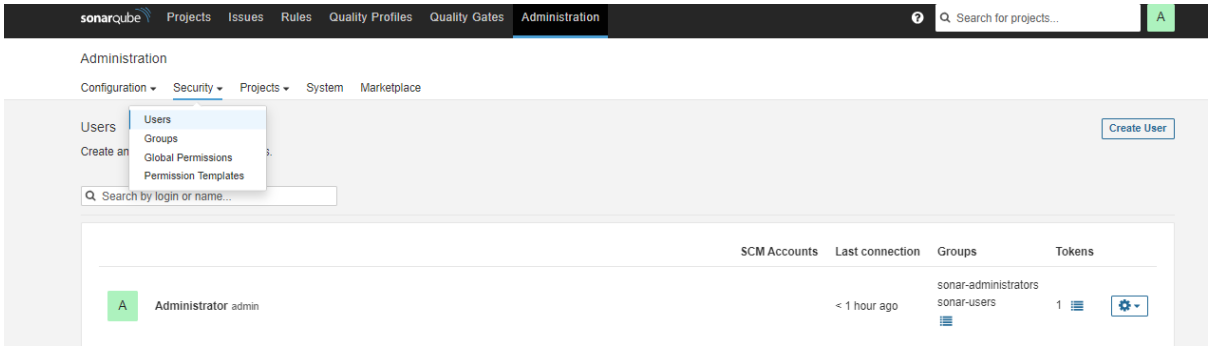
16.20.2



Apply & save the setting

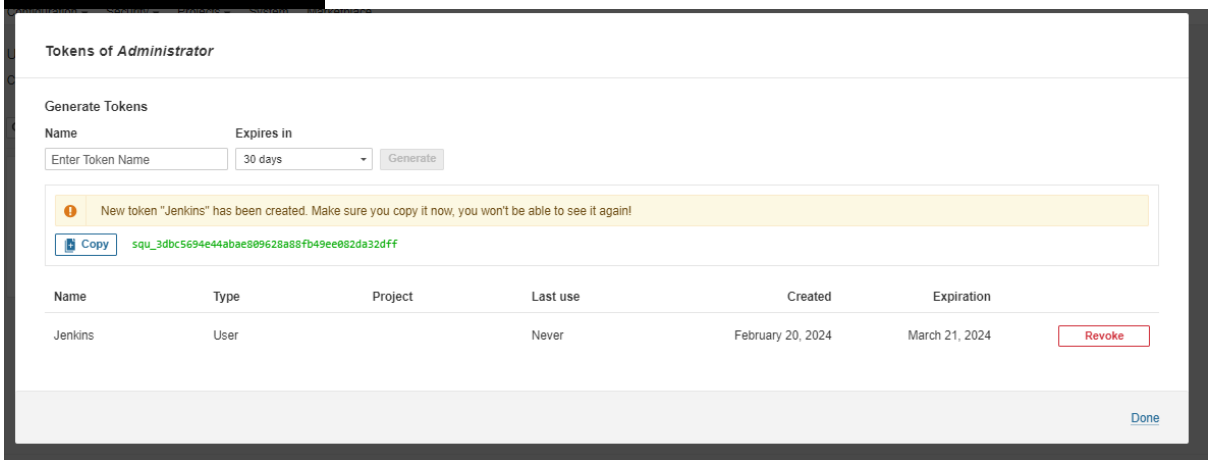
SonarQube token

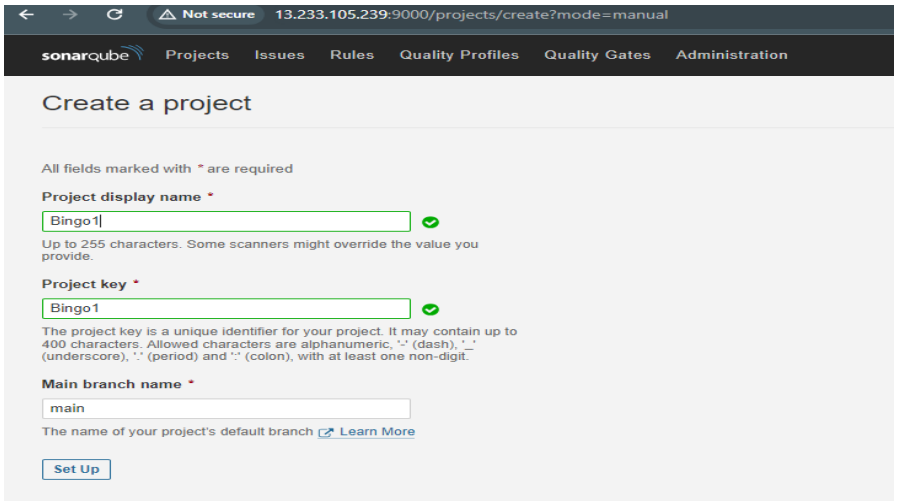
Click on Administration & security & user then click on right 3 dot to create new token



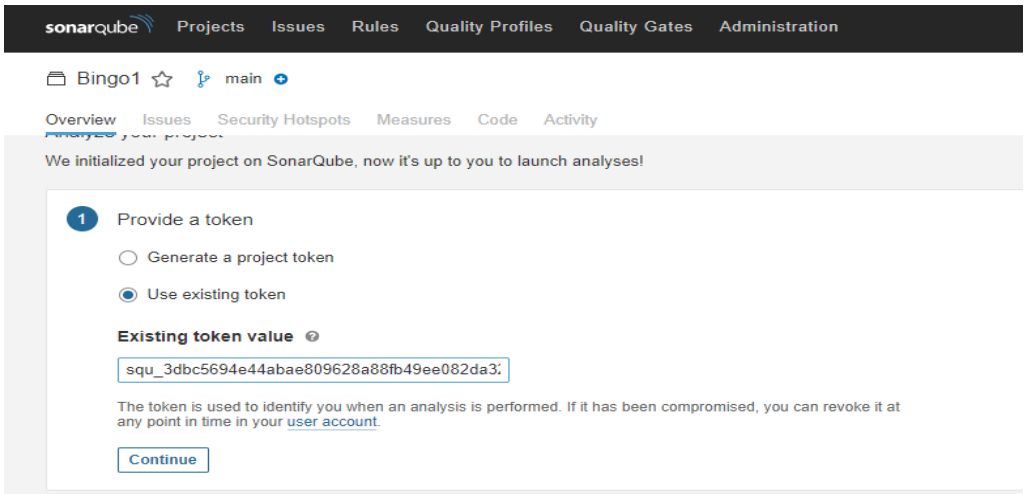
Sonar token for Jenkins integration

squ_3dbc5694e44abae809628a88fb49ee082da32dff



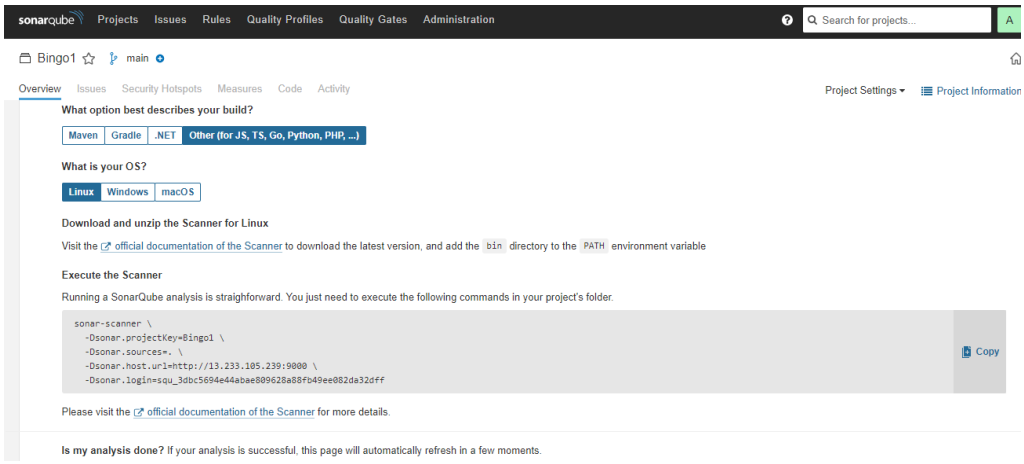


Click on setup , click on locally , provide the token

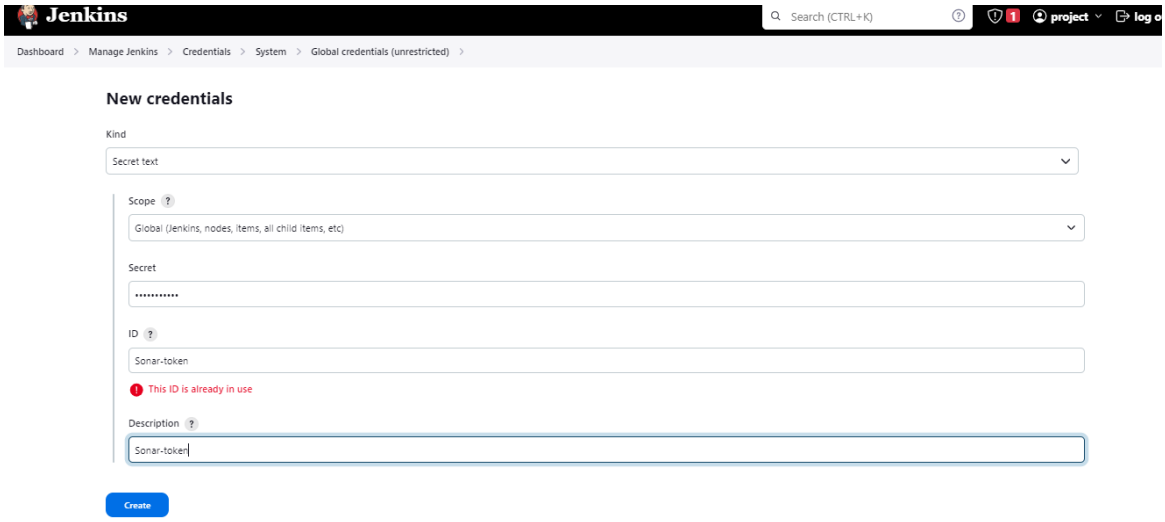



```
sonar-scanner \
  -Dsonar.projectKey=Bingo1 \
  -Dsonar.sources=. \
```

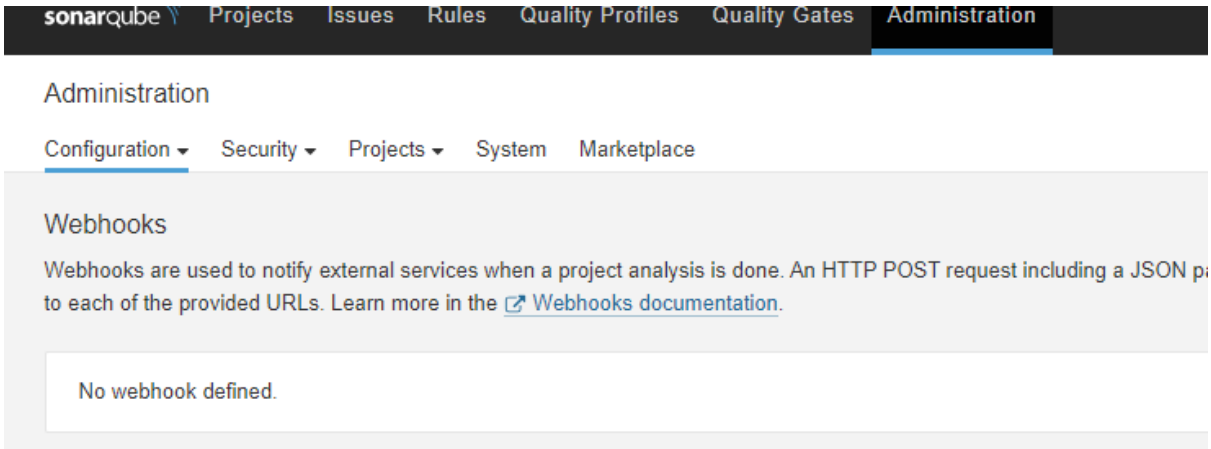
-Dsonar.host.url=http://13.233.105.239:9000 \
-Dsonar.login=squ_3dbc5694e44abae809628a88fb49ee082da32dff

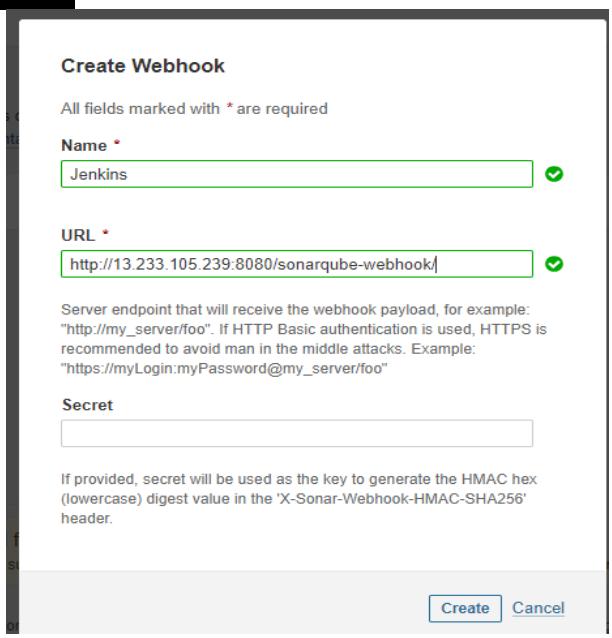


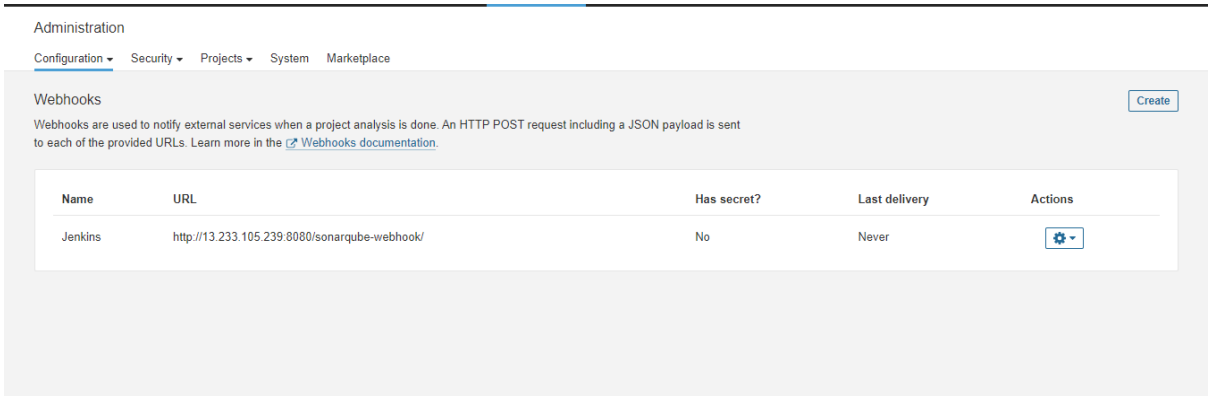
Jenkins in order to add SonarQube token in Jenkins credentials sections



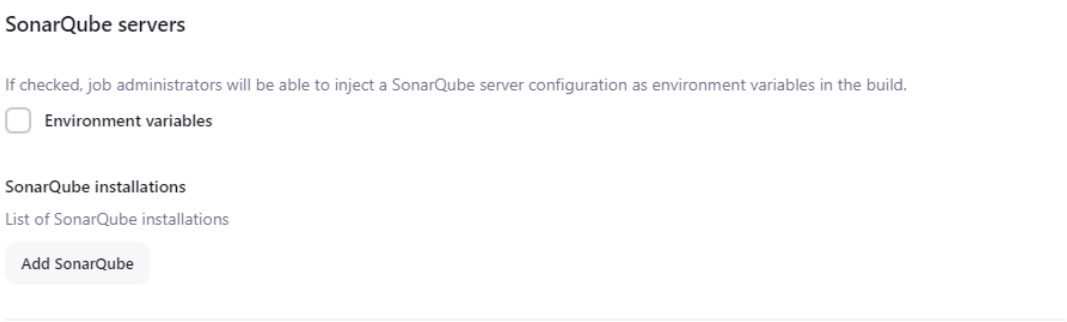
Quality Gate



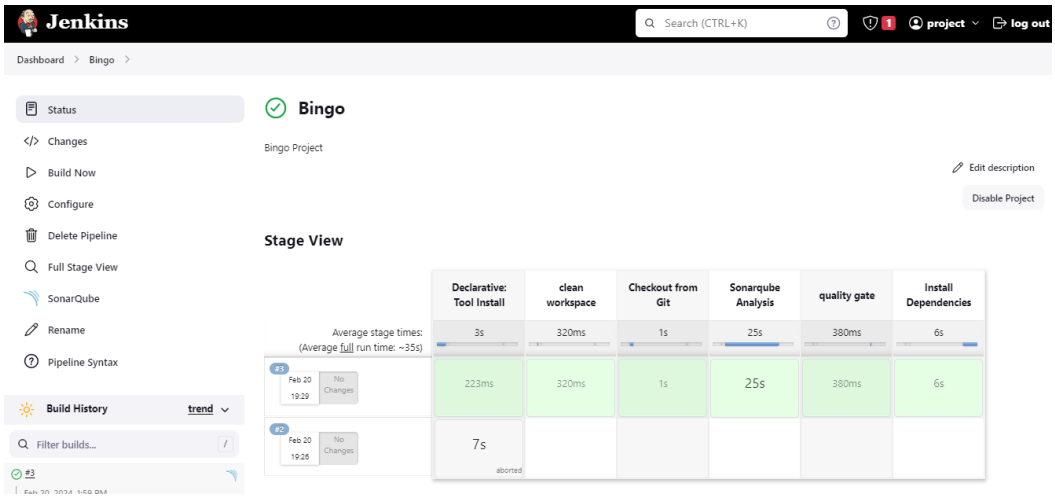




Configure the sonarqube server

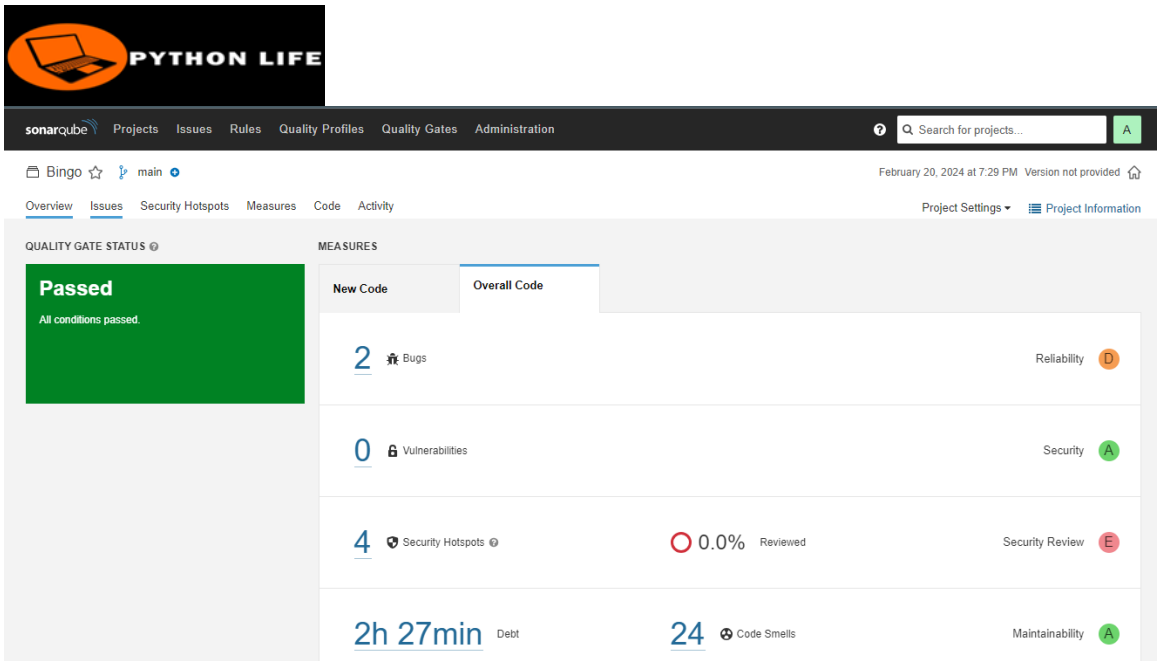


Create new pipeline in for bingo project




```
pipeline{
  agent any
  tools{
    jdk 'jdk17'
    nodejs 'node16'
  }
  #environment {
    #SCANNER_HOME=tool 'sonar-scanner'
  #}
  stages {
    stage('clean workspace'){
      steps{
        cleanWs()
```

```
        }
    }
    stage('Checkout from Git'){
        steps{
            git branch: 'main', url: 'https://github.com/Milky19/bingo1.git'
        }
    }
    stage("Sonarqube Analysis "){
        steps{
            withSonarQubeEnv('sonar-server') {
                sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Bingo \
                -Dsonar.projectKey=Bingo '''
            }
        }
    }
    stage("quality gate"){
       steps {
            script {
                waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
            }
        }
    }
    stage('Install Dependencies') {
        steps {
            sh "npm install"
        }
    }
  }
}
```



Add remaining tools checks in our pipelines , Now go configure → Pipeline and add this stage to your pipeline and build.

OWASP Dependency-Check is an open-source tool that identifies project dependencies and checks if there are any known, publicly disclosed, vulnerabilities in those dependencies. It is part of the Open Web Application Security Project (OWASP) and is designed to help organizations and developers identify and mitigate security risks related to third-party libraries and components used in their software projects.

Key aspects of OWASP Dependency-Check:

1. **Identification of Vulnerabilities:**
    - Dependency-Check analyses project dependencies, including libraries, frameworks, and components.
    - It checks these dependencies against a database of known vulnerabilities, including the National Vulnerability Database (NVD) and other sources.
2. **Automatic Scanning:**
    - Dependency-Check can be integrated into the development process and CI/CD pipelines.
    - It automatically scans dependencies to identify and report vulnerabilities, allowing for early detection and remediation.5
3. **Multiple Language Support:**
    - Dependency-Check supports multiple programming languages and package managers, making it versatile for a wide range of projects.
    - It can analyze dependencies in Java, JavaScript, .NET, Python, Ruby, and more.
4. **Integration with Build Tools:**

- It integrates with popular build tools such as Maven, Gradle, npm, and others, making it easy to incorporate into existing development workflows.

5. **Usefulness:**
    - **Early Vulnerability Detection:** Dependency-Check enables developers to identify and address security issues early in the development lifecycle.
    - **Continuous Monitoring:** It supports continuous monitoring of dependencies, helping to keep software secure over time as new vulnerabilities are discovered.
6. **Applicability:**
    - **Web Applications:** Especially relevant for web applications where security vulnerabilities in dependencies can lead to serious security issues.
    - **Any Software with Dependencies:** Applicable to any software project that relies on third-party libraries or components.
7. **Complementary to SCA Practices:**
    - Dependency-Check is part of Software Composition Analysis (SCA) practices, focusing on the security aspects of third-party software components.
8. **Open Source and Actively Maintained:**
    - As an open-source project, Dependency-Check benefits from community contributions and is actively maintained to keep the vulnerability database up-to-date.

```
stage('OWASP FS SCAN') {
        steps {
            dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit',
odcInstallation: 'DP'
            dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
        }
    }
    stage('TRIVY FS SCAN') {
        steps {
            sh "trivy fs . > trivyfs.txt"
        }
    }
```

Trivy is an open-source vulnerability scanner designed for containerized environments. Here's a breakdown of its key aspects:

What is Trivy?

Vulnerability Scanner: Trivy is primarily used as a vulnerability scanner, focusing on container images. It helps identify security vulnerabilities within the components (libraries, packages, dependencies) of containerized applications.

Container Security: Trivy is specifically tailored for container security, making it a valuable tool in DevSecOps (Development, Security, and Operations) workflows. It's commonly used with container orchestration platforms like Kubernetes and containerization technologies like Docker.

Why use Trivy?

Early Detection of Vulnerabilities: Trivy aids in the early detection of vulnerabilities in container images. By scanning container images during the development and CI/CD (Continuous Integration/Continuous Deployment) processes, security issues can be addressed before deployment, reducing the risk of exposing vulnerabilities in production.

Automation: Trivy supports automation, allowing developers and DevOps teams to integrate vulnerability scanning seamlessly into their CI/CD pipelines. This ensures that security checks are part of the automated build and deployment processes.

Where is Trivy used?

Containerized Environments: Trivy is primarily used in containerized environments, where applications are packaged along with their dependencies into containers. This includes popular containerization platforms such as Docker and container orchestration systems like Kubernetes.

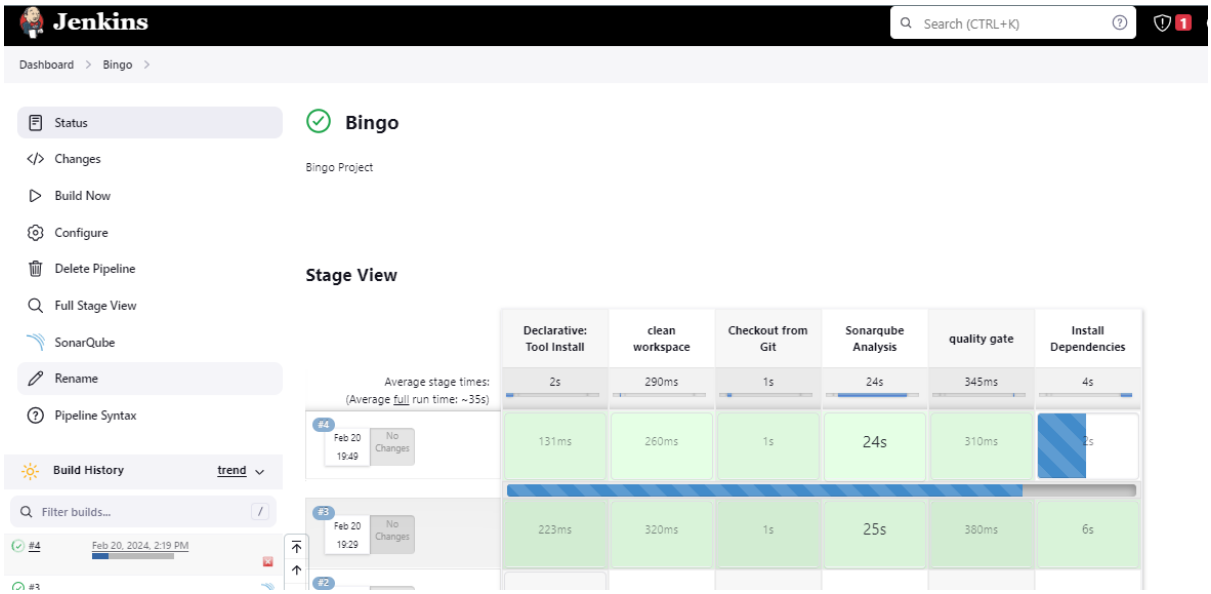
DevSecOps Pipelines: Trivy is integrated into DevSecOps pipelines to automate security checks throughout the software development lifecycle. It helps teams identify and remediate vulnerabilities early in the development process.

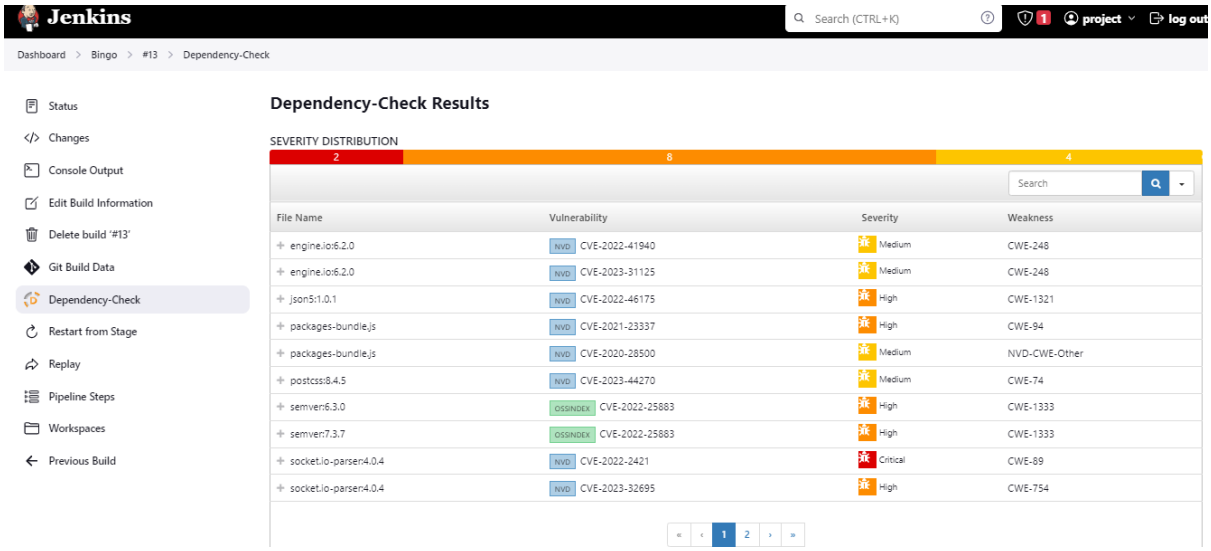
Security Audits: Trivy can be used for regular security audits of container images in production environments. This ongoing monitoring helps ensure that the deployed containers remain secure against the latest vulnerabilities.

```
        }
        stage('OWASP FS SCAN') {
            steps {
                dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit'
                dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
            }
        }
        stage('TRIVY FS SCAN') {
            steps {
                sh "trivy fs . > trivyfs.txt"
            }
        }

    }
}
```

Build now the pipeline





Go to docker hub & created a access token to integrate with Jenkins for Docker build & docker Push ,& docker pull to deploy

https://hub.docker.com/

Add docker credentials in credentials

Username – ##########
Password - ##########

Add this stage to Pipeline Script

```
stage("Docker Build & Push") {
        steps {
            script {
                withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                    sh "docker build -t bingo ."
                    sh "docker tag bingo hanvitha/bingo:latest "
                    sh "docker push hanvitha/bingo:latest "
                }
            }
        }
    }
    stage("TRIVY"){
        steps{
            sh "trivy image hanvitha/bingo:latest > trivyimage.txt"
        }
    }
```

You will see the output below, with a dependency trend.

Now Run the container to see if the bingo app is live with adding the below stage

PYTHON LIFE

Dashboard > Bingo > #15

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#15'
- Git Build Data
- Dependency-Check
- Restart from Stage
- Replay
- Pipeline Steps

Build #15 (Feb 20, 2024, 4:37:28 PM)

Keep this build forever

Add description

Started 27 min ago
Took 4 min 31 sec

Started by user project

Revision: c59d69b04b39674ec254225a3d00960861980027
Repository: https://github.com/Aseemakram19/bingo1.git

- refs/remotes/origin/main

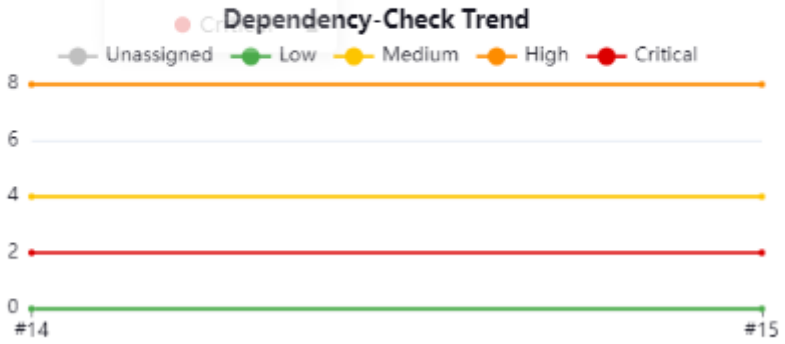
Stage View

	Declarative: Tool Install	clean workspace	Checkout from Git	Sonarqube Analysis	quality gate	Install Dependencies	OWASP FS SCAN	TRIVY FS SCAN	Docker Build & Push	TRIVY	Deploy to container
Average stage times: (Average full run time: ~4min 31s)	219ms	403ms	1s	24s	351ms	8s	1min 50s	6s	1min 19s	1min 10s	1s
#15 Feb 20 22:07 No Changes	158ms	283ms	2s	23s	322ms	8s	1min 50s	7s	45s	1min 10s	1s
#14 Feb 20 21:59 No Changes	281ms	523ms	1s	25s	381ms	8s	1min 51s	6s	1min 53s		

Unassigned 0
Low 0
Medium 4
High 8

Edit description

Disable Project



PYTHON LIFE

Dashboard > Bingo > #15 > Allocate node : Start > Workspace > Workspace

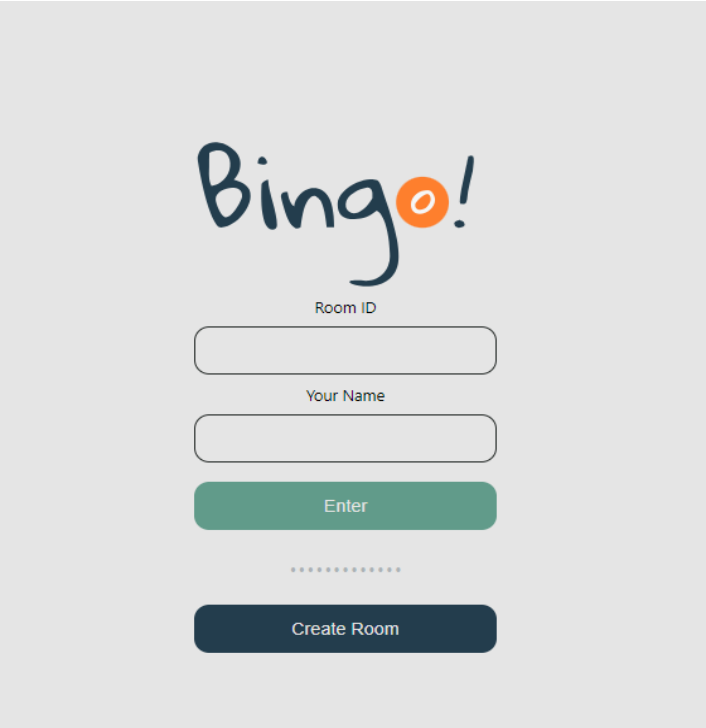
- Up
- Status
- Console Output
- Workspace

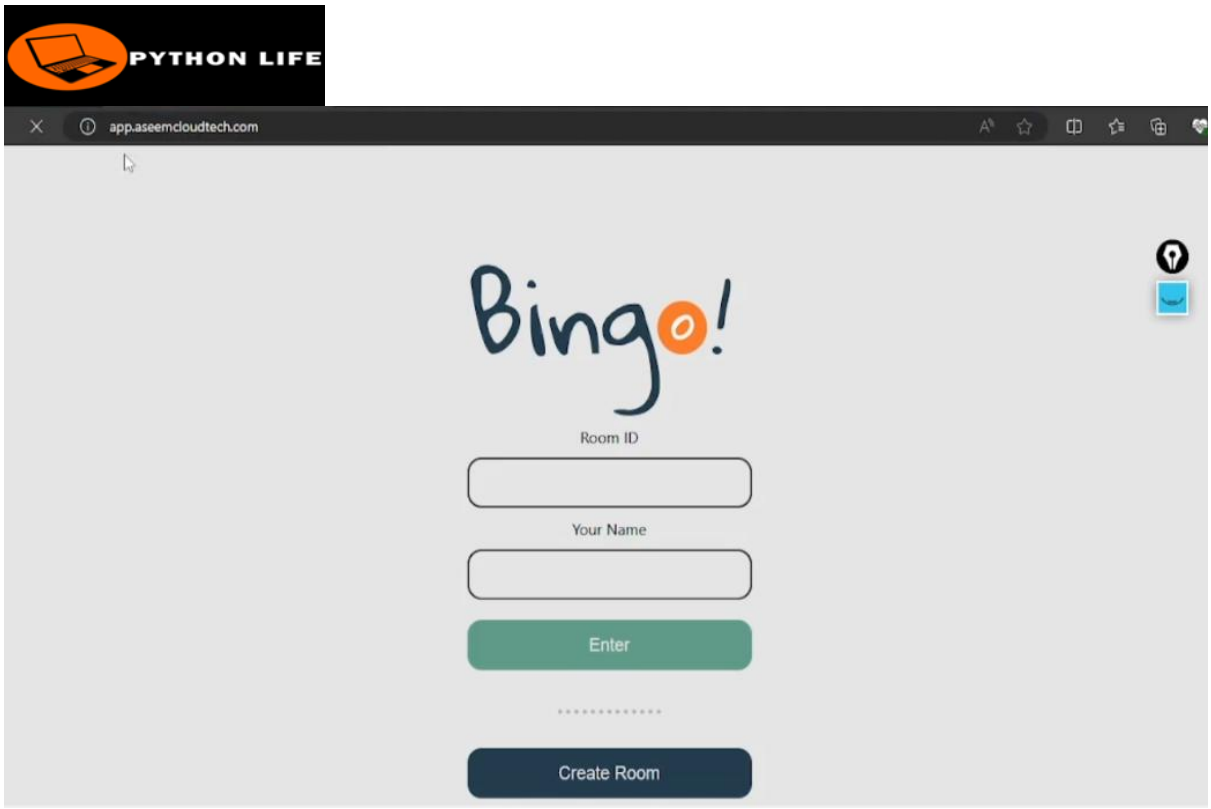
Workspace

/

.git		
.scannerwork		
components		
node_modules		
pages		
public		
styles		
utils		
.eslintrc.json	Feb 20, 2024, 4:37:31 PM	40 B
.gitignore	Feb 20, 2024, 4:37:31 PM	343 B
dependency-check-report.xml	Feb 20, 2024, 4:39:54 PM	4.16 MB
dev.html	Feb 20, 2024, 4:37:31 PM	31 B
docker-compose.yml	Feb 20, 2024, 4:37:31 PM	172 B
Dockerfile	Feb 20, 2024, 4:37:31 PM	504 B
next.config.js	Feb 20, 2024, 4:37:31 PM	118 B
package.json	Feb 20, 2024, 4:37:31 PM	549 B
package-lock.json	Feb 20, 2024, 4:38:03 PM	203.05 KB
README.md	Feb 20, 2024, 4:37:31 PM	1.84 KB
trivyfs.txt	Feb 20, 2024, 4:40:01 PM	8.69 KB
trivyimage.txt	Feb 20, 2024, 4:41:58 PM	2.27 MB
Untitled Diagram.drawio	Feb 20, 2024, 4:37:31 PM	12.38 KB

(all files in zip)






```
#!/bin/bash
# 1. jenkins installation on ubuntu
sudo apt update -y
sleep 5
sudo apt install fontconfig openjdk-17-jre -y
sleep 15
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update -y
sleep 5
sudo apt-get install jenkins -y
sleep 10
sudo systemctl enable jenkins
sudo systemctl start jenkins
```

sleep 10

 # 2. Install Docker & SONARQUBE AS A CONTAINER IN THE EC2 INSTANCE

sudo apt-get update -y

sudo apt-get install docker.io -y

sleep 5

sudo usermod -aG docker ubuntu

sudo usermod -aG docker jenkins

newgrp docker

sleep 5

sudo chmod 777 /var/run/docker.sock

sleep 10

docker run -d --name sonar -p 9000:9000 sonarqube:lts-community

sleep 5

 # 3. Install trivy ON THE INSTANCE

sudo apt-get install wget apt-transport-https gnupg lsb-release -y

sleep 5

wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | gpg --dearmor | sudo tee /usr/share/keyrings/trivy.gpg > /dev/null

echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" | sudo tee -a /etc/apt/sources.list.d/trivy.list

sudo apt-get update -y

sleep 5

sudo apt-get install trivy -y

sleep 5

 # 4. Terraform Installation

sudo apt-get update && sudo apt-get install -y gnupg software-properties-common

wget -O- https://apt.releases.hashicorp.com/gpg | \

gpg --dearmor | \

sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg

gpg --no-default-keyring \

--keyring /usr/share/keyrings/hashicorp-archive-keyring.gpg \

--fingerprint

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \

https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \

sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update

sudo apt-get install terraform -y
```

 # 5. Install kubectl

```
sudo apt update

sleep 5

sudo apt install curl -y

sleep 5

curl -LO https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl

sleep 5

sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

kubectl version --client

sleep 10
```

 # 6. Install AWS CLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

sleep 5

sudo apt-get install unzip -y

sleep 5

unzip awscliv2.zip

sudo ./aws/install
```

#7. eksctl installation

```
curl --silent --location
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -
s)_amd64.tar.gz" | tar xz -C /tmp

sleep 5

sudo mv /tmp/eksctl /usr/local/bin

sleep 5
```

#Status of installation

eksctl version

kubectl version --client

terraform -v

docker -version

aws --version

java --version